

EX.NO : 01

DATE :

DOWNLOAD, INSTALL AND EXPLORE THE FEATURES OF NUMPY, SCIPY, JUPYTER,STATSMODELS AND PANDAS PACKAGES.

AIM:

Download, install and explore the features of NumPy, SciPy, Jupyter, Statsmodels and Pandas Packages.

ALGORITHM:

1. Download and Install Packages:

Define a function `install_package(package_name)` to install packages using pip.

Call `install_package("numpy")` to install NumPy.

Call `install_package("scipy")` to install SciPy.

Call `install_package("jupyter")` to install Jupyter.

Call `install_package("statsmodels")` to install Statsmodels.

Call `install_package("pandas")` to install Pandas.

Explore Features:

2. NumPy:

Import NumPy using `import numpy as np`.

Use NumPy functions to create arrays and perform mathematical operations.

3. SciPy:

Import SciPy using `import scipy`.

Explore different modules of SciPy such as optimization, integration, interpolation, etc.

4. Jupyter:

Import Jupyter using `import jupyter`.

Start a Jupyter notebook server using the command `jupyter notebook`.

5. Statsmodels:

Import Statsmodels using `import statsmodels.api as sm`.

Explore features related to statistical models, tests, and data exploration.

6. Pandas:

Import Pandas using `import pandas as pd`.

Explore Pandas DataFrame and Series, and perform data manipulation and analysis.

Example Code:

Include example code snippets for each package to demonstrate its features.

For NumPy, include an example of array creation and mathematical operations.

For SciPy, include an example of linear algebra operations or other relevant functionalities.

For Statsmodels, include an example of a statistical model or test.

For Pandas, include an example of creating a DataFrame and performing data analysis.

7. Documentation and Resources:

Provide links to documentation and additional resources for each package.

8. Testing and Validation:

Test the code snippets to ensure they work as expected.

Validate the installation and functionality of each package.

Make adjustments as necessary to address any issues encountered during testing.

9. Revision and Updates:

Periodically review and update the lab manual to incorporate new features, versions, or best practices for each package.

Incorporate feedback from users to improve the clarity and effectiveness of the manual.

This algorithm outlines the process of creating a lab manual for downloading, installing, and exploring the features of NumPy, SciPy, Jupyter, Statsmodels, and Pandas packages. It provides a structured approach to help users learn and utilize these powerful tools for scientific computing and data analysis in Python.

PROCEDURE:

To download Anaconda, follow these steps:

Go to the Anaconda Website: Open your web browser and navigate to the Anaconda website. You can find it at <https://www.anaconda.com/products/distribution>.

Choose the Right Version: Anaconda is available in two editions: Anaconda Individual Edition and Anaconda Team Edition. For individual users, the Individual Edition is typically the right choice. Click on "Individual Edition" to proceed.

Select Your Operating System: Anaconda supports Windows, macOS, and Linux. Choose the version appropriate for your operating system by clicking on the respective download button.

Download the Installer: Once you select your operating system, the website will display a download button. Click on it to start downloading the Anaconda installer.

Install Anaconda:

Windows: Double-click the downloaded installer (.exe file) to start the installation process. Follow the on-screen instructions, and when prompted, choose whether to install Anaconda for just the current user or all users. You can also choose the installation location and whether to add Anaconda to your system PATH environment variable.

macOS: Double-click the downloaded .pkg file to begin the installation. Follow the on-screen instructions, and when prompted, agree to the license agreement and choose the installation location.

Linux: Open a terminal and navigate to the directory where the downloaded installer (.sh file) is located. Run the installer using the command `bash Anaconda3-<version>-Linux-x86_64.sh` (replace <version> with the version number). Follow the prompts to complete the installation.

Verify the Installation: Once the installation is complete, you can verify it by opening a terminal or command prompt and running the command `conda --version`. This should display the version of conda, which is the package manager included with Anaconda.

Update Anaconda (Optional): It's a good idea to update Anaconda after installation to ensure you have the latest packages and bug fixes. You can do this by running the command `conda update --all` in a terminal or command prompt.

The standard packaging tools are all designed to be used from the command line.

The following command will install the latest version of a module and its dependencies from the Python Package Index:

```
python -m pip install SomePackage
```

It's also possible to specify an exact or minimum version directly on the command line. When using comparator operators such as `>`, `<` or some other special character which get interpreted by shell, the package name and the version should be enclosed within double quotes:

```
python -m pip install SomePackage==1.0.4 # specific version  
python -m pip install "SomePackage>=1.0.4" # minimum version
```

Normally, if a suitable module is already installed, attempting to install it again will have no effect. Upgrading existing modules must be requested explicitly:

```
python -m pip install --upgrade SomePackage
```

Install packages:

With pip, a package is installed. To install the `<package-name>` generic package, run this command:

```
C:> pip install <package-name>
```

To install the `<package-name>` generic package, run this command:

```
C:> conda install <package-name>
```

To install particular version of the package

```
C:>conda install <package-name> = 1.11.0
```

To install multiple packages at a time, by listing all their names:

```
C:> conda install <package-name1> <package-name2>
```

To update,

```
C:>conda update <package-name>
```

To uninstall packages,

```
C:>conda remove <package-name>
```

To install data science packages using Anaconda, you can use the conda command, which is Anaconda's package manager. Here's how you can install some popular data science packages:

NumPy, SciPy, and pandas:

These packages are fundamental for data manipulation, scientific computing, and data analysis.

```
conda install numpy scipy pandas
```

Matplotlib and Seaborn:

Matplotlib is a plotting library, and Seaborn is a statistical data visualization library based on Matplotlib.

```
conda install matplotlib seaborn
```

scikit-learn:

scikit-learn is a machine learning library that provides simple and efficient tools for data mining and data analysis.

```
conda install scikit-learn
```

Statsmodels:

Statsmodels is a Python module that provides classes and functions for the estimation of many different statistical models, as well as for conducting statistical tests, and statistical data exploration.

```
conda install statsmodels
```

Jupyter Notebook:

Jupyter Notebook is an interactive computing environment that allows you to create and share documents containing live code, equations, visualizations, and narrative text.

```
conda install jupyter
```

TensorFlow or PyTorch (Optional, for deep learning):

TensorFlow and PyTorch are popular deep learning frameworks.

```
conda install tensorflow
```

or

conda install pytorch torchvision torchaudio cudatoolkit=10.2 -c pytorch

The second command installs PyTorch with GPU support. Adjust the cudatoolkit version according to your CUDA installation.

OUTPUT :

```
Command Prompt

C:\Program Files\Python312\Scripts>pip install pandas
Defaulting to user installation because normal site-packages is not writeable
Collecting pandas
  Downloading pandas-2.2.2-cp312-cp312-win_amd64.whl.metadata (19 kB)
Collecting numpy>=1.26.0 (from pandas)
  Downloading numpy-1.26.4-cp312-cp312-win_amd64.whl.metadata (61 kB)
----- 61.0/61.0 kB 81.3 kB/s eta 0:00:00
Collecting python-dateutil>=2.8.2 (from pandas)
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting pytz>=2020.1 (from pandas)
  Downloading pytz-2024.1-py2.py3-none-any.whl.metadata (22 kB)
Collecting tzdata>=2022.7 (from pandas)
  Downloading tzdata-2024.1-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting six>=1.5 (from python-dateutil>=2.8.2->pandas)
  Downloading six-1.16.0-py2.py3-none-any.whl.metadata (1.8 kB)
Download pandas-2.2.2-cp312-cp312-win_amd64.whl (11.5 MB)
----- 11.5/11.5 MB 222.4 kB/s eta 0:00:00
Download numpy-1.26.4-cp312-cp312-win_amd64.whl (15.5 MB)
----- 15.5/15.5 MB 448.5 kB/s eta 0:00:00
Download python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
----- 229.0/229.9 kB 299.3 kB/s eta 0:00:00
Download pytz-2024.1-py2.py3-none-any.whl (505 kB)
----- 505.5/505.5 kB 689.7 kB/s eta 0:00:00
Download tzdata-2024.1-py2.py3-none-any.whl (345 kB)
----- 345.4/345.4 kB 78.9 kB/s eta 0:00:00
Download six-1.16.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: pytz, tzdata, six, numpy, python-dateutil, pandas
  WARNING: The script f2py.exe is installed in 'C:\Users\ELCOT\AppData\Roaming\Python\Python312\Scripts' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed numpy-1.26.4 pandas-2.2.2 python-dateutil-2.9.0.post0 pytz-2024.1 six-1.16.0 tzdata-2024.1

C:\Program Files\Python312\Scripts>pip install scipy
Defaulting to user installation because normal site-packages is not writeable
Collecting scipy
  Downloading scipy-1.13.0-cp312-cp312-win_amd64.whl.metadata (60 kB)
----- 60.6/60.6 kB 52.8 kB/s eta 0:00:00
Requirement already satisfied: numpy<2.3, >=1.22.4 in c:\users\elcot\appdata\roaming\python\python312\site-packages (from scipy) (1.26.4)
Download scipy-1.13.0-cp312-cp312-win_amd64.whl (45.9 MB)
----- 45.9/45.9 MB 204.3 kB/s eta 0:00:00
Installing collected packages: scipy
Successfully installed scipy-1.13.0

C:\Program Files\Python312\Scripts>
```

Class Performance	
Viva	
Record	
Total	

RESULT:

Thus the Download, install and explore the features of NumPy, SciPy, Jupyter, Statsmodels and Pandas packages are done with Anaconda Navigator.

EX.NO : 02
DATE :

**PRE-PROCESS THE DATASET TO REMOVE MISSING VALUES, OUTLIERS
AND TO TRANSFORM AND DISCRETIZE THE DATASET**

AIM:

Pre-process the dataset to remove missing values, outliers and to transform and Discretize the dataset

ALGORITHM:

1. Outlier is a data point that stands out significantly from the rest of the data.
2. It can be an extremely high or low value compared to the other observations in a dataset.
3. Outliers can be caused by measurement errors, natural variations in the data, or even unexpected discoveries.
4. Outlier detection is a method used to find unusual or abnormal data points in a set of information.
5. For Normal Distributions
6. Use empirical relations of Normal distribution.
7. The data points that fall below $\text{mean} - 3 * (\text{sigma})$ or above $\text{mean} + 3 * (\text{sigma})$ are outliers, where mean and sigma are the average value and standard deviation of a particular column.
8. Use Inter-Quartile Range (IQR) proximity rule.
9. The data points that fall below $Q1 - 1.5 \text{ IQR}$ or above the third quartile $Q3 + 1.5 \text{ IQR}$ are outliers, where $Q1$ and $Q3$ are the 25th and 75th percentile of the dataset, respectively. IQR represents the inter quartile range and is given by $Q3 - Q1$.

PROGRAM:

```
import pandas as pd
from numpy import nan
import matplotlib.pyplot as plt
import seaborn as sns
# Step 1: Load dataset
dataset = pd.read_csv('https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv',
                      header=None)

# Step 2: Replace 0 with NaN in specific columns
dataset[[1, 2, 3, 4, 5]] = dataset[[1, 2, 3, 4, 5]].replace(0, nan)

# Step 3: Print missing values per column
print("Missing Values Per Column:")
print(dataset.isnull().sum())

# Step 4: Plot missing values using seaborn
plt.figure(figsize=(10, 6))
sns.barplot(x=dataset.columns.astype(str), y=dataset.isnull().sum().values)
plt.title('Missing Values per Column')
```

```
plt.xlabel('Column Index')
plt.ylabel('Number of Missing Values')
plt.show()
```

```
# Step 5: Show dataset summary with NaNs
print("\nDataset Summary (with missing
values):")
print(dataset.describe())
```

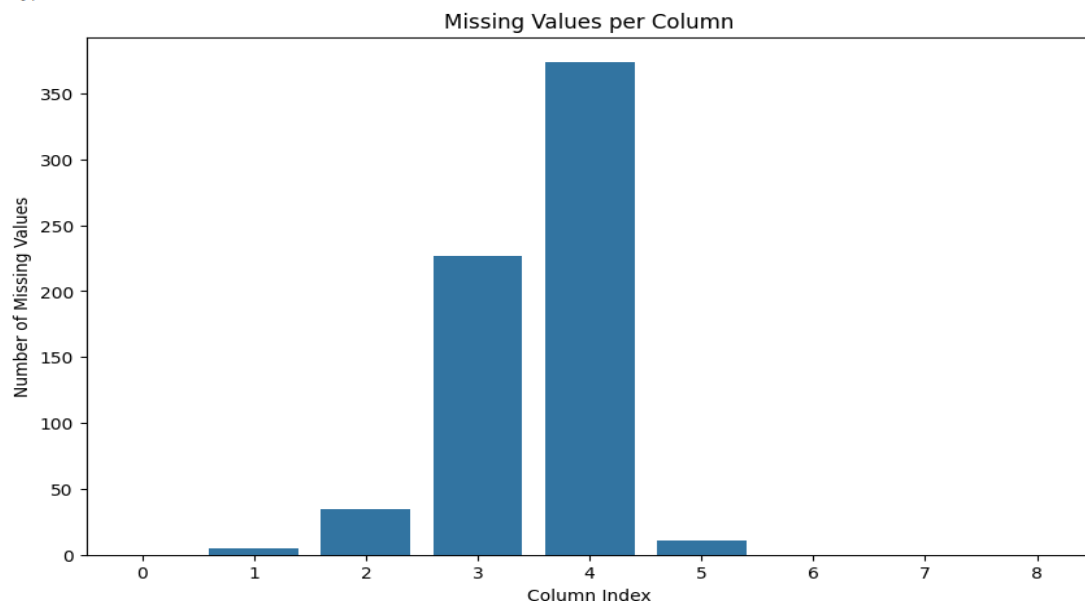
```
# Step 6: Show first 20 rows
print("\nFirst 20 Rows:")
print(dataset.head(20))
```

```
# Step 7: Drop rows with any missing values
dataset.dropna(inplace=True)
```

```
# Step 8: Show shape after removing
missing rows
print("\nDataset shape after removing rows
with missing values:")
print(dataset.shape)
```

OUTPUT:

```
Missing Values Per Column:
0      0
1       5
2      35
3     227
4     374
5      11
6       0
7       0
8       0
dtype: int64
```



Dataset Summary (with missing values):

```

count    0      1      2      3      4      5 \
mean    3.845052 121.686763 72.405184 29.153420 155.548223 32.457464
std     3.369578 30.535641 12.382158 10.476982 118.775855 6.924988
min     0.000000 44.000000 24.000000 7.000000 14.000000 18.200000
25%     1.000000 99.000000 64.000000 22.000000 76.250000 27.500000
50%     3.000000 117.000000 72.000000 29.000000 125.000000 32.300000
75%     6.000000 141.000000 80.000000 36.000000 190.000000 36.600000
max    17.000000 199.000000 122.000000 99.000000 846.000000 67.100000

count    6      7      8
mean    0.471876 33.240885 0.348958
std     0.331329 11.760232 0.476951
min     0.078000 21.000000 0.000000
25%     0.243750 24.000000 0.000000
50%     0.372500 29.000000 0.000000
75%     0.626250 41.000000 1.000000
max     2.420000 81.000000 1.000000

```

First 20 Rows:

```

0      0      1      2      3      4      5      6      7      8
0      6  148.0  72.0  35.0   NaN  33.6  0.627  50  1
1      1   85.0  66.0  29.0   NaN  26.6  0.351  31  0
2      8  183.0  64.0   NaN   NaN  23.3  0.672  32  1
3      1   89.0  66.0  23.0  94.0  28.1  0.167  21  0
4      0  137.0  40.0  35.0 168.0  43.1  2.288  33  1
5      5  116.0  74.0   NaN   NaN  25.6  0.201  30  0
6      3   78.0  50.0  32.0  88.0  31.0  0.248  26  1
7     10  115.0   NaN   NaN   NaN  35.3  0.134  29  0
8      2  197.0  70.0  45.0 543.0  30.5  0.158  53  1
9      8  125.0  96.0   NaN   NaN   NaN  0.232  54  1
10     4  110.0  92.0   NaN   NaN  37.6  0.191  30  0
11    10  168.0  74.0   NaN   NaN  38.0  0.537  34  1
12    10  139.0  80.0   NaN   NaN  27.1  1.441  57  0
13     1  189.0  60.0  23.0 846.0  30.1  0.398  59  1
14     5  166.0  72.0  19.0 175.0  25.8  0.587  51  1
15     7  100.0   NaN   NaN   NaN  30.0  0.484  32  1
16     0  118.0  84.0  47.0 230.0  45.8  0.551  31  1
17     7  107.0  74.0   NaN   NaN  29.6  0.254  31  1
18     1  103.0  30.0  38.0  83.0  43.3  0.183  33  0
19     1  115.0  70.0  30.0  96.0  34.6  0.529  32  1

```

Dataset shape after removing rows with missing values:
(392, 9)

Class Performance	
Viva	
Record	
Total	

RESULT:

Thus the outliers are detected executing the python code.

EXNO: 3

DATE :

IMPLEMENT THE CLASSIFICATION TECHNIQUES: DECISION TREE AND NAIVE BAYES ALGORITHMS AND COMPARE ITS PERFORMANCE.

AIM :

To Implement the classification techniques: Decision Tree and Naive Bayes algorithms and compare its performance.

DECISION TREE

ALGORITHM :

Step-1: Begin the tree with the root node, says S, which contains the complete dataset. Step-2: Find the best attribute in the dataset using Attribute Selection Measure (ASM). Step-3: Divide the S into subsets that contains possible values for the best attributes.

Step-4: Generate the decision tree node, which contains the best attribute.

Step-5: Recursively make new decision trees using the subsets of the dataset created in step - 3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

PROGRAM :

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.datasets import load_iris

data = load_iris()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42 )
```

```

clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred) conf_matrix =
confusion_matrix(y_test, y_pred) class_report =
classification_report(y_test, y_pred) print(f"Accuracy:
{accuracy}")
print("Confusion Matrix:")
print(conf_matrix)
print("Classification Report:")
print(class_report)

```

OUTPUT :

```

➡ Accuracy: 1.0
Confusion Matrix:
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
Classification Report:

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

NAIVE BAYES ALGORITHM

ALGORITHM :

Bayes' Theorem finds the probability of an event occurring given the probability of another event that has already occurred. Bayes' theorem is stated mathematically as the following equation:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

where A and B are events and $P(B) \neq 0$

- Basically, we are trying to find probability of event A, given the event B is true. Event B is also termed as evidence.
- $P(A)$ is the priori of A (the prior probability, i.e. Probability of event before evidence is seen). The evidence is an attribute value of an unknown instance(here, it is event B).
- $P(B)$ is Marginal Probability: Probability of Evidence.
- $P(A|B)$ is a posteriori probability of B, i.e. probability of event after evidence is seen.
- $P(B|A)$ is Likelihood probability i.e the likelihood that a hypothesis will come true based on the evidence. Now, with regards to our dataset, we can apply Bayes' theorem in following way:

$$P(y|X) = \frac{P(X|y)P(y)}{P(X)}$$

where, y is class variable and X is a dependent feature vector (of size n) where:

$$X = (x_1, x_2, x_3, \dots, x_n)$$

PROGRAM :

```
from sklearn.datasets import load_iris iris
= load_iris()
X = iris.data y
= iris.target
```

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=1)

from sklearn.naive_bayes import GaussianNB

gnb = GaussianNB()

gnb.fit(X_train, y_train)

y_pred = gnb.predict(X_test)

from sklearn import metrics

print("Gaussian Naive Bayes model accuracy (in %):", metrics.accuracy_score(y_test, y_pred)
* 100)

```

OUTPUT :



```

Gaussian Naive Bayes model accuracy (in %): 95.0

```

Class Performance	
Record	
Viva	
Total	

RESULT :

Thus the program is executed to determine the decision tree and Naïve Bayes algorithms

AIM:

To Implement K-Means clustering algorithms.

ALGORITHM:

Step1 – Specify the number of clusters, K, need to be generated by this algorithm.

Step 2 – Randomly select K data points and assign each data point to a cluster. Classify the data based on the number of data points.

Step3– Compute the cluster centroids.

Step4–Keep iterating the following until the optimal centroid is determined which is the assignment of data points to the clusters that are not changing any more

–First, the sum of squared distance between data points and centroids would be computed.

–Now , we have to assign each data point to the cluster that is closer than other cluster (centroid).

–At last compute the centroids for the clusters by taking the average of all data points of that cluster.

K-means follows Expectation – Maximization approach to solve the problem. The Expectation-step is used for assigning the data points to the closest cluster and the Maximization-step is used for computing the centroid of each cluster.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
data=np.array([[185,72],[170,56],[168,60],[179,68],[182,72],[188,77]])
initial_centroids = np.array([data[0],data[1]])
def euclidean_distance(a, b):
    return np.linalg.norm(a-b)
def assign_clusters(data, centroids):
    clusters = {}
    for x in data:
        distances = [euclidean_distance(x,centroid) for centroid in centroids]
        cluster = distances.index(min(distances))
        clusters[cluster] = []
        clusters[cluster].append(x)
    return clusters
def recompute_centroids(clusters):
    new_centroids=[]
```

```

    return new_centroids
centroids = initial_centroids
for i in range(2):
    clusters = assign_clusters(data , centroids)
    centroids = recompute_centroids(clusters)
    print (f'Iteration {i+1}:')
    print (f'Centroids: {centroids}')
    colors = ['r', 'b']
for cluster in clusters:
    for point in clusters[cluster]:
        plt.scatter (point[0],point[1],color=colors[cluster])
for centroid in centroids:
    plt.scatter(centroid[0],centroid[1],color='k',marker='x')
plt.xlabel('Height(cm)')
plt.ylabel('Weight(kg)')
plt.title('K-Means Clustering(K=2)')
plt.show()

```

OUTPUT:

```

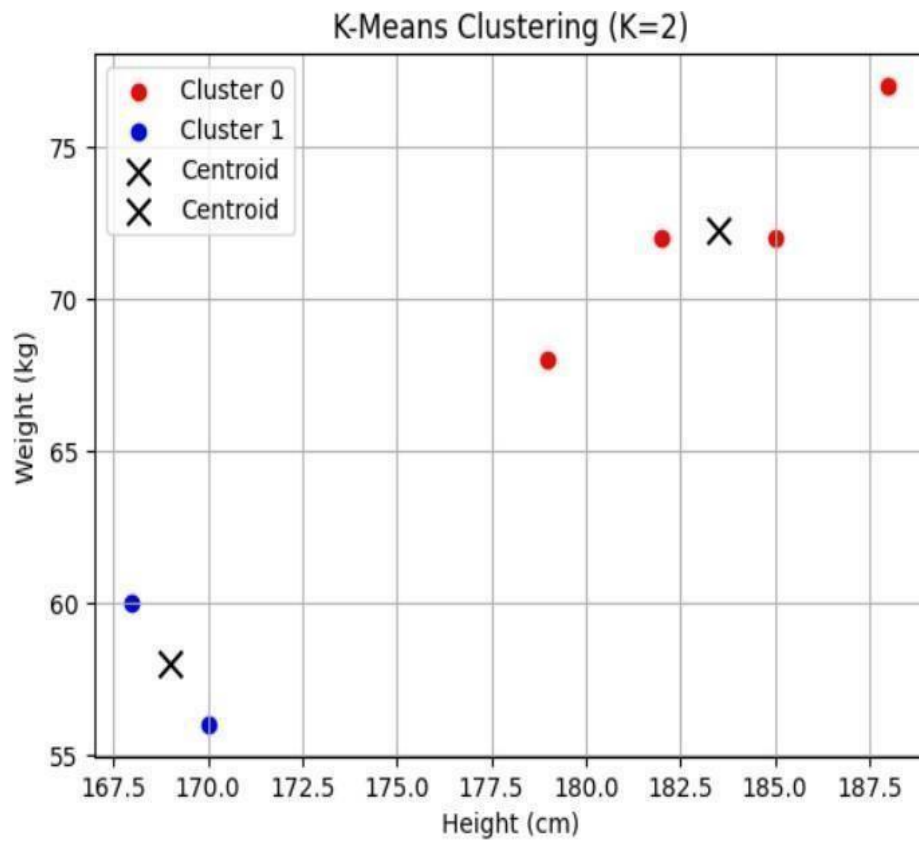
➡ Iteration 1:
Centroids:
[[183.5  72.25]
 [169.   58.  ]]

```

```

Iteration 2:
Centroids:
[[183.5  72.25]
 [169.   58.  ]]

```



Class Performance	
Record	
Viva	
Total	

RESULT:

Thus the program is executed to find the clusters using K Means algorithm.

EX.NO : 4(b)

DATE :

IMPLEMENT K-MEDOIDS CLUSTERING ALGORITHMS

.

AIM :

To Implement K-Medoids clustering algorithms.

ALGORITHM:

1. Choose K points from the dataset and assign them to K number of clusters. These K points act as initial medoids. Each object is taken into one cluster.
2. The distance between the initial medoids (points) and other points (non-medoid) is calculated using either of the distance metrics like Euclidian distance, or Manhattan distance. etc.
3. The non – medoid points are assigned to that particular cluster to which its distance to the medoid point is minimum.
4. Now the total cost is calculated which is the sum of distances from other points to the medoid point within a cluster.
5. A random new non – medoid objects is selected and is swapped with the initial medoid object r, and the cost is recalculated.

If costs < costr , then the swap becomes permanent.

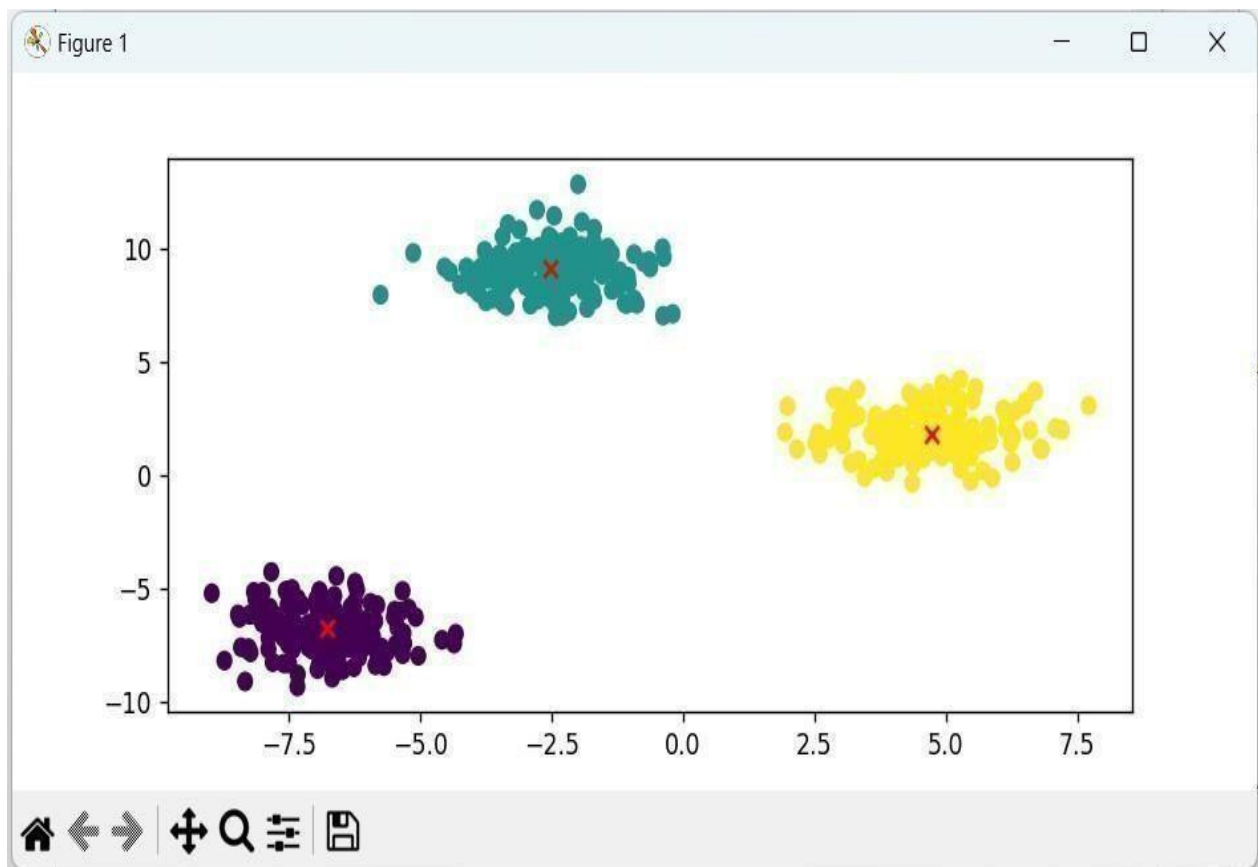
6. The steps from 2 to 6 are repeated until there is no change in cost.

PROGRAM:

```
from sklearn_extra.cluster import KMedoids
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

X, y = make_blobs(n_samples=500, centers=3, random_state=42)
kmedoids = KMedoids(n_clusters=3, random_state=42)
kmedoids.fit(X)
plt.figure(figsize=(7.5, 3.5))
plt.scatter(X[:, 0], X[:, 1], c=kmedoids.labels_, cmap='viridis')
plt.scatter(kmedoids.cluster_centers_[0],
            kmedoids.cluster_centers_[0, 1], marker='x', color='red')
plt.show()
```

OUTPUT:



Class Performance	
Record	
Viva	
Total	

RESULT:

Thus the program is executed to define the clusters using K Medoids algorithm.

EX.NO:05

DATE :

IMPLEMENT APRIORI ALGORITHM TO DERIVE ASSOCIATION RULES.

AIM:

To Implement Apriori algorithm to derive association rules.

ALGORITHM:

1. Set a minimum value for support and confidence. ...
2. Extract all the subsets having higher value of support than minimum threshold.
3. Select all the rules from the subsets with confidence value higher than minimum threshold.
4. Order the rules by descending order of Lift.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from apyori import apriori
#Load the dataset
store_data = pd.read_csv('store_data.csv', header = None)
#Print the first few rows of the dataset
print(store_data.head())
#Convert the dataset into a list of lists
records = []
for i in range (0,500):
records.append ([str(store_data.values [i,j])
for j in range (0,20)])
#Apply Apriori algorithm
association_rules=apriori(records,min_support=0.0045,min_confidence=0.2,min_lift=3, min_length=2)
association_results = list (association_rules)
#Print the number of association rules
print (len(association_results))
#Print the first association rule
print (association_results[0])
#Loop through the association results and print
each rule for item in association_results:
    #First index of the inner list – Contains base item and add item
    pair = item[0]
    items=[x for x in pair]
    print("Rule:"+items[0]+"->" +items[1])
    #Second index of the inner list
```

```

print("Support:" + str(item[1]))
# of the third index of the inner list
print("Confidence:" + str(item[2][0][2]))
print("Lift: " + str(item[2][0][3]))
print("=====")

```

OUTPUT:

```

Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/ybkal/DSclab/5.apriori.py =====
0      1      2      ...      17      18      1
9
0      shrimp    almonds    avocado    ...    frozen smoothie    spinach    olive oi
1
1      burgers    meatballs    eggs    ...    NaN    NaN    Na
N
2      chutney    NaN    NaN    ...    NaN    NaN    Na
N
3      turkey    avocado    NaN    ...    NaN    NaN    Na
N
4      mineral water    milk    energy bar    ...    NaN    NaN    Na
N

[5 rows x 20 columns]
743
RelationRecord(items=frozenset({'burgers', 'almonds'}), support=0.008, ordered_s
tatistics=[OrderedStatistic(items_base=frozenset({'almonds'}), items_add=frozens
et({'burgers'}), confidence=0.4, lift=5.128205128205129)])
Rule: burgers -> almonds
Support: 0.008
Confidence: 0.4
Lift: 5.128205128205129
=====
Rule: soup -> almonds
Support: 0.006
Confidence: 0.3
Lift: 3.9473684210526314
=====
Rule: avocado -> olive oil
Support: 0.008
Confidence: 0.23529411764705882
Lift: 3.9215686274509807
=====
Rule: avocado -> soup
Support: 0.012

```

Class Performance	
Record	
Viva	
Total	

RESULT:

Thus the program is executed by implementing Apriori algorithm to derive association rules.

EX.NO:06

DATE:

CLASSIFY THE INPUT DATASET USING SUPPORT VECTOR MACHINE (SVM)

AIM:

To Implement Classify the input dataset using Support Vector Machine (SVM) algorithms.

ALGORITHM:

Step1: Install Required Libraries

Hyperplane

A hyperplane is a decision boundary which separates between given set of data points having different class labels. The SVM classifier separates data points using a hyperplane with the maximum amount of margin. This hyperplane is known as the maximum margin hyperplane and the linear classifier it defines is known as the maximum margin classifier.

Support Vectors

Support vectors are the sample data points, which are closest to the hyperplane. These datapoints will define the separating line or hyperplane better by calculating margins.

Margin

A margin is a separation gap between the two lines on the closest data points. It is calculated as the perpendicular distance from the line to support vectors or closest data points. In SVMs , we try to maximize this separation gap so that we get maximum margin.

SVM searches for the maximum margin hyperplane in the following 2 step process –

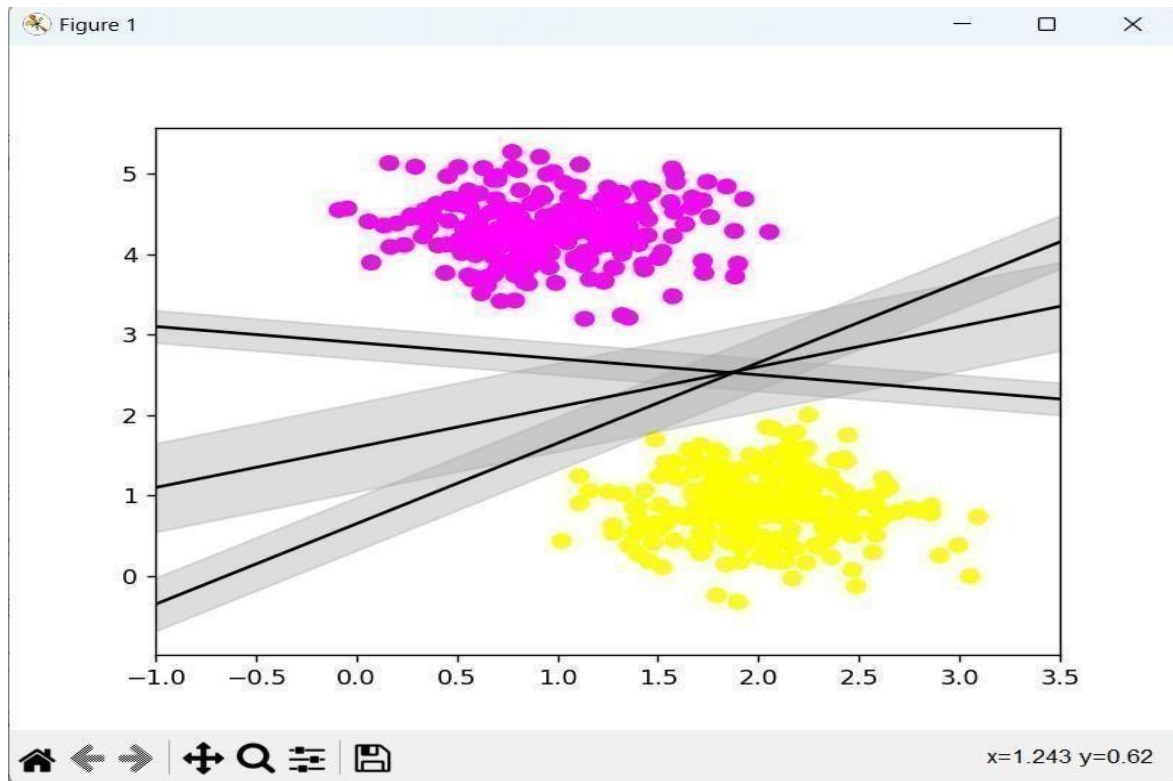
1. Generate hyperplanes which segregates the classes in the best possible way. There are many hyperplanes that might classify the data. We should look for the best hyperplane that represents the largest separation, or margin, between the two classes.
2. So, we choose the hyperplane so that distance from it to the support vectors on each side is maximized. If such a hyperplane exists, it is known as the maximum margin hyperplane and the linear classifier it defines is known as a maximum margin classifier.

PROGRAM:

```
import pandas as pd
import numpy as np
from sklearn.datasets import
make_blobs#creating datasets X containing
n_samples # Y containing two classes
X, Y =make_blobs (n_samples = 500, centers = 2,
                  random_state = 0 , cluster_std = 0.40)
import matplotlib.pyplot as plt
#creating linspace between -1 to 3.5 xfit = np.linspace (-1,3.5)
#plotting scatter
plt.scatter(X[:,0] , X[:,1] , c=Y , s=50 , cmap = 'spring')
form ,b,d in [(1,0.65,0.33) , (0.5,1.6,0.55) , (-0.2,2.9,0.2)]:
yfit = m * xfit + b
plt.plot(xfit,yfit,'-k')
```

```
plt.fill_between(xfit, yfit-d, yfit+d, edgecolor='none',
color='#AAAAAA', alpha=0.4)
plt.xlim(-1,3.5); plt.show()
```

OUTPUT:



Class Performance	
Record	
Viva	
Total	

RESULT:

Thus the code is executed for SVM classifier.

EX.NO:7

DATE:

DEMONSTRATE CLUSTERING APPROACH USING KOHONEN'S SELF-ORGANIZING MAP

AIM:

To Implement Kohonen's self-organizing map

Kohonen's Self-Organizing Map (SOM) is a type of artificial neural network that is used for clustering and visualizing high-dimensional data. It does this by creating a two-dimensional map that maintains the topological properties of the original data. Here's a step-by-step demonstration of using Kohonen's SOM for clustering:

ALGORITHM:

1. Install Required Libraries
2. Import Libraries
3. Load the Dataset
4. Preprocess the Data: Normalize the data to ensure that all features contribute equally.
5. Initialize and Train the SOM: Create a SOM instance and train it with the dataset.
6. Visualize the Results: Plot the SOM grid and the clusters.
7. Analyze the Clusters: The U-Matrix (Unified Distance Matrix) visualizes the distances between the neurons in the SOM.

Training:

Step1: Initialize the weights w_{ij} random value may be assumed. Initialize the learning rate α .

Step 2: Calculate squared Euclidean distance.

$$D(j) = \sum (w_{ij} - x_i)^2 \quad \text{where } i = 1 \text{ to } n \text{ and } j = 1 \text{ to } m$$

Step3: Find index J , when $D(j)$ is minimum that will be considered as winning index.

Step4: For each j within a specific neighborhood of j and for all i , calculate the new weight.

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha [x_i - w_{ij}(\text{old})]$$

Step5: Update the learning rule by using :

$$\alpha(t+1) = 0.5 * t$$

Step6: Test the Stopping Condition.

PROGRAM:

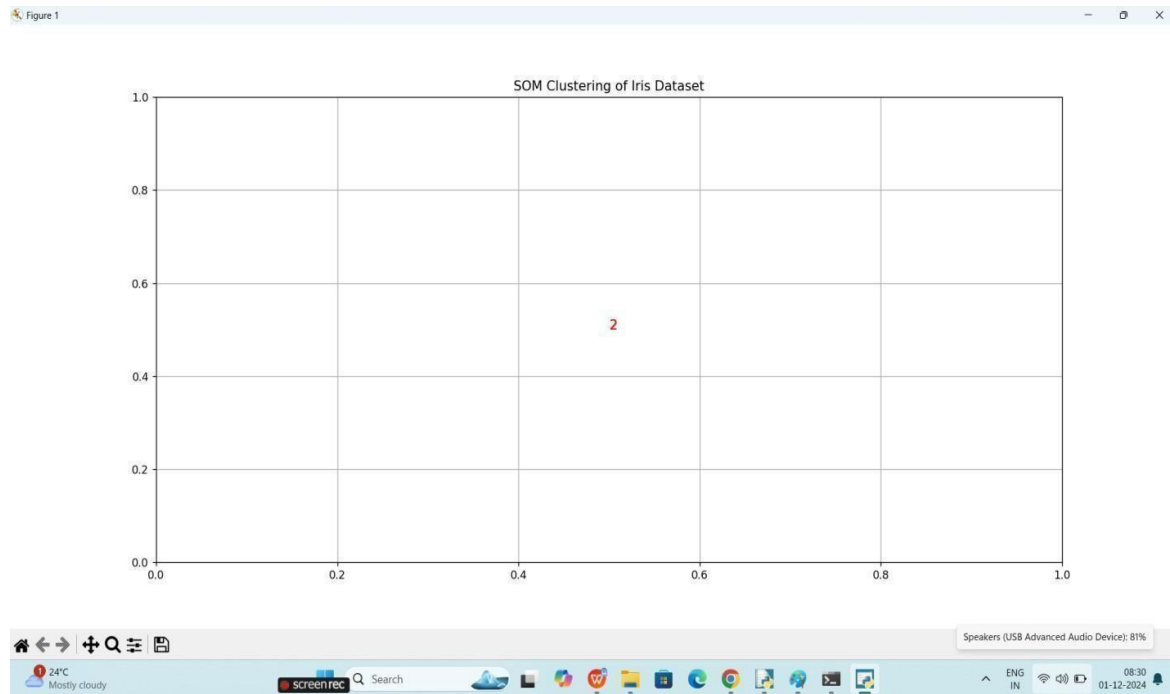
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from minisom import MiniSom
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
df = pd.DataFrame(data.data, columns = data.feature_names)
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df)
som = MiniSom(x=10, y=10, input_len = df_scaled.shape[1], sigma=1.0, learning_rate=0.5)
```

```

som.random_weights_init (df_scaled)
som.train_random (df_scaled,500) #Train with 100 iterations
plt.figure (figsize=(10,10))
for i , x in enumerate (df_scaled):
    w = som.winner (x) # Get the winning node for each data point
    plt.text(w[0]+0.5,w[1]+0.5,str(data.target[i]),color=plt.cm.rainbow(data.target[i]/2), fontdict={'size':
12}) plt.grid() plt.title ('SOM Clustering of Iris Dataset')
plt.show()

```

OUTPUT:



Class Performance	
Record	
Viva	
Total	

RESULT:

Thus the code is executed for Kohonen's self loop on Iris dataset.

EX.NO:08

DATE:

**IMPLEMENT PRINCIPAL COMPONENT ANALYSIS (PCA) TECHNIQUE TO
REDUCE THE DIMENSIONALITY OF THE DATASET**

AIM:

To Implement Principal Component Analysis (PCA) technique to reduce the dimensionality of the dataset

ALGORITHM:

Step-1: Import necessary libraries

Step-2: Load the dataset

Step-3: Standardize the features

Step-4: Applying Principal Component Analysis

Step-5: Checking Co – relation between features after PCA

PROGRAM::

```
from sk_learn import datasets
import pandas as pd
from sk_learn.preprocessing import StandardScaler # to standardize the features
from sklearn.decomposition import PCA # to apply PCA
import seaborn as sns #top lot the heat maps
import matplotlib.pyplot as plt
import numpy as np
#Load the Dataset
iris = datasets.load_iris()
#convert the dataset into a pandas data frame
df = pd.DataFrame(iris['data'], columns = iris['feature_names'])
#display the head (first 5 rows) of the dataset
print(df.head())
#Standardize the features
#Create an object of StandardScaler which is present in sklearn.preprocessing
scalar = StandardScaler()
scaled_data = pd.DataFrame(scalar.fit_transform(df)) # scaling the data
print(scaled_data)
#Applying PCA
#Taking no.of Principal Components as 3
pca = PCA(n_components = 3)
pca.fit(scaled_data)
data_pca = pca.transform(scaled_data)
data_pca = pd.DataFrame(data_pca, columns = ['PCAC1', 'PCAC2', 'PCAC3'])
```

```

print(data_pca.head())
#Checking Co – relation between features after PCA data_pca_corr = data_pca.corr()
sns.heatmap (data_pca_corr,annot = True)
plt.show()

```

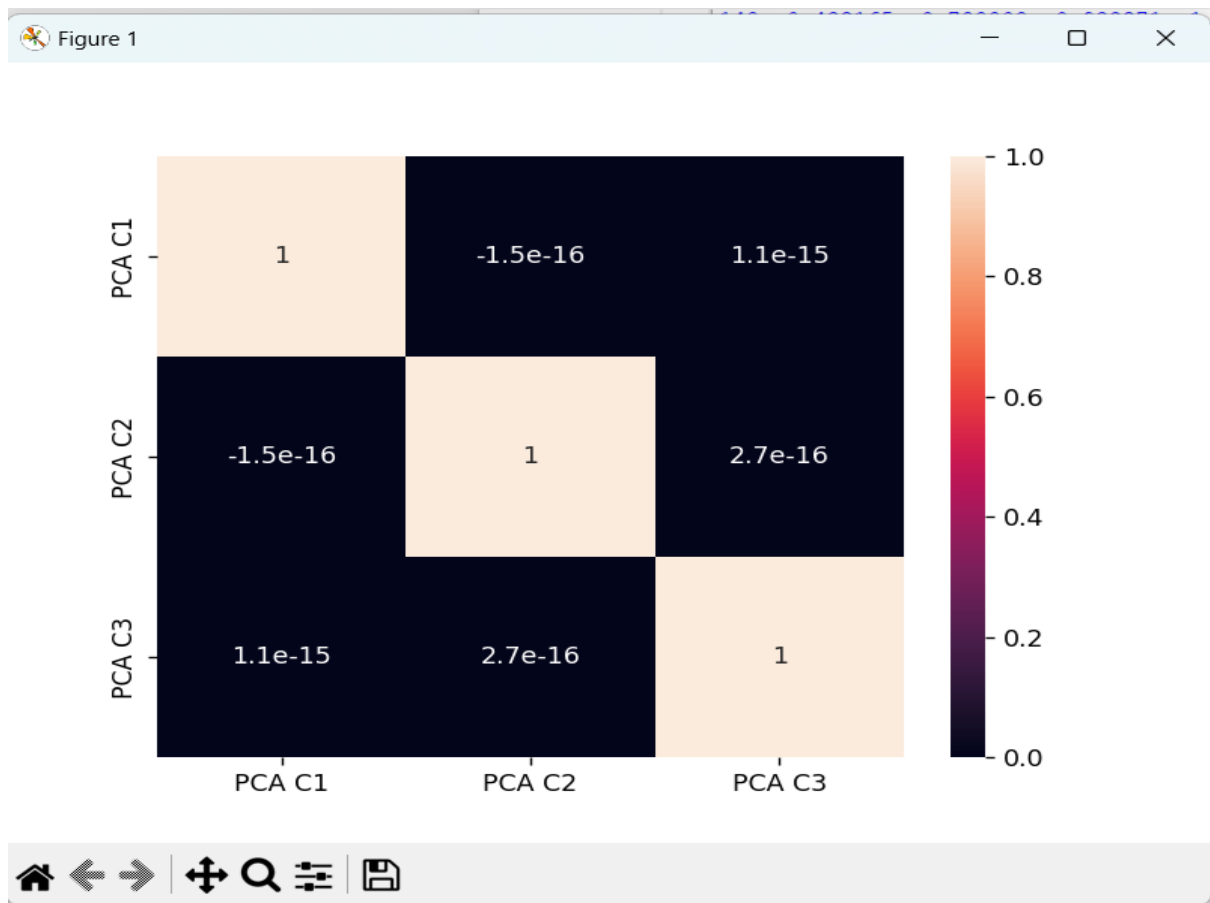
OUTPUT:

```

Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/ybkal/DSclab/8.1 pca 1.py =====
      sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0              5.1             3.5             1.4             0.2
1              4.9             3.0             1.4             0.2
2              4.7             3.2             1.3             0.2
3              4.6             3.1             1.5             0.2
4              5.0             3.6             1.4             0.2
      0      1      2      3
0 -0.900681  1.019004 -1.340227 -1.315444
1 -1.143017 -0.131979 -1.340227 -1.315444
2 -1.385353  0.328414 -1.397064 -1.315444
3 -1.506521  0.098217 -1.283389 -1.315444
4 -1.021849  1.249201 -1.340227 -1.315444
..
145  1.038005 -0.131979  0.819596  1.448832
146  0.553333 -1.282963  0.705921  0.922303
147  0.795669 -0.131979  0.819596  1.053935
148  0.432165  0.788808  0.933271  1.448832
149  0.068662 -0.131979  0.762758  0.790671

[150 rows x 4 columns]
      PCA C1      PCA C2      PCA C3
0 -2.264703  0.480027 -0.127706
1 -2.080961 -0.674134 -0.234609
2 -2.364229 -0.341908  0.044201
3 -2.299384 -0.597395  0.091290
4 -2.389842  0.646835  0.015738

```



Class Performance	
Record	
Viva	
Total	

RESULT:

The above heat map clearly depicts that there is no correlation between various obtained principal components (PC1, PC2, and PC3). Thus, we have moved from higher dimensional feature space to a lower-dimensional feature space while ensuring that there is no correlation between the so obtained PCs is minimum. Hence, we have accomplished the objectives of PCA.

EX.NO:
DATE:

Visualizing Data using Tableau

Aim:

To visualize and analyse data interactively by creating charts and dashboards using Tableau.

Procedure:

1. Open Tableau Desktop or Tableau Public.
2. Click **Connect** and choose a data source (for example, “Sample – Superstore” which comes with Tableau).
3. Drag **State** (or any geographic field) to the view to automatically generate a map.
4. Drag **Sales** (or any measure) to **Color** to show intensity.
5. Drag **Profit** (or any measure) to **Size** to vary the size of the marks.
6. Rename the sheet as “Sales Map”.
- (Optional) Create additional charts (like a bar chart of Sales by Category) in new worksheets.
7. Click **Dashboard** → **New Dashboard**, then drag your sheets into the dashboard to combine them.
8. Add filters, legends, and titles to customize.
9. Click **File** → **Save to Tableau Public** or **Export** → **Image/PDF** to save your work.

Output:



Class Performance	
Record	
Viva	
Total	

Result:

Thus, the visualization of data using Tableau was successfully carried out and an interactive dashboard was created.

EX.NO:
DATE:

Data Visualization Using TensorFlow

Aim:

To visualize training progress, loss, accuracy curves, and other metrics of a neural network model using TensorFlow's built-in visualization tool, TensorBoard.

Algorithm:

1. Install TensorFlow using pip install tensorflow.
2. Import TensorFlow, Keras, NumPy, and Matplotlib libraries.
3. Load dataset (e.g., MNIST) using tf.keras.datasets.
4. Split dataset into training and test sets.
5. Preprocess data: normalize pixel values and flatten images if needed.
6. Visualize sample data using matplotlib.pyplot.imshow().
7. Define a Sequential model with Dense layers (hidden layer ReLU, output layer Softmax).
8. Compile model with optimizer = adam, loss = sparse_categorical_crossentropy, metrics = accuracy.
9. Train model on training data for specified epochs (e.g., 5).
10. Evaluate model on test data and print accuracy.
11. Visualize training history (accuracy and loss) using Matplotlib.

Program:

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.callbacks import TensorBoard
import datetime

# Load and preprocess data
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
train_images = train_images / 255.0
test_images = test_images / 255.0

# Define model
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Create log directory with timestamp
log_dir = "logs/fit/" + datetime.datetime.now().strftime("%Y%m%d-%H%M%S")
tensorboard_callback = TensorBoard(log_dir=log_dir, histogram_freq=1)
```

```
# Train model with TensorBoard callback
model.fit(train_images, train_labels,
          epochs=5,
          validation_data=(test_images, test_labels),
          callbacks=[tensorboard_callback])
```

```
# After training, run this in terminal to launch TensorBoard:
# tensorboard --logdir logs/fit
```

Output:

```
Epoch 1/5
1875/1875 ██████████ 7s 3ms/step - accuracy: 0.8771 - loss: 0.4387
Epoch 2/5
1875/1875 ██████████ 11s 3ms/step - accuracy: 0.9632 - loss: 0.1214
Epoch 3/5
1875/1875 ██████████ 5s 3ms/step - accuracy: 0.9758 - loss: 0.0792
Epoch 4/5
1875/1875 ██████████ 7s 3ms/step - accuracy: 0.9829 - loss: 0.0563
Epoch 5/5
1875/1875 ██████████ 5s 3ms/step - accuracy: 0.9869 - loss: 0.0442
313/313 ██████████ 1s 2ms/step - accuracy: 0.9727 - loss: 0.0869
Test accuracy: 0.9769999980926514
```

Class Performance	
Record	
Viva	
Total	

Result:

Thus, TensorBoard was successfully used to visualize the training metrics and data of a TensorFlow model.

EX.NO:
DATE:

Reading data from text files

AIM

Reading data from text files, Excel and the web and exploring various commands for doing descriptive analytics on the Iris data set using python.

ALGORITHM

Step 1: Loads the Iris dataset from a text file, an Excel file, and the web.

Step 2: Displays basic information about each dataset.

Step 3: Calculates descriptive statistics (mean, median, min, max, etc.) for the Iris dataset from the text file.

Step 4: Visualizes the distribution of features (sepal length, sepal width, petal length, petal width) using histograms.

PROGRAM

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import requests

iris_text = pd.read_csv(r'C:\Users\ybk\DS\lab\IRIS dataset\iris.txt', sep=',', header=None,
names=['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'class'])
iris_excel = pd.read_csv(r'C:\Users\ybk\DS\lab\IRIS dataset\iris.csv')
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
iris_web = pd.read_csv(url, header=None, names=['sepal_length', 'sepal_width', 'petal_length',
'petal_width', 'class'])
print("Iris dataset from text file:")
print(iris_text.head())
print("\nIris dataset from Excel file:")
print(iris_excel.head())
print("\nIris dataset from the web:")
print(iris_web.head())
print("\nDescriptive statistics for Iris dataset from text file:")
print(iris_text.describe())
plt.figure(figsize=(10, 6))
plt.hist(iris_text['sepal_length'], bins=20, color='blue', alpha=0.7, label='Sepal Length')
plt.hist(iris_text['sepal_width'], bins=20, color='green', alpha=0.7, label='Sepal Width')
plt.hist(iris_text['petal_length'], bins=20, color='red', alpha=0.7, label='Petal Length')
plt.hist(iris_text['petal_width'], bins=20, color='orange', alpha=0.7, label='Petal Width')
plt.xlabel('Feature Value')
plt.ylabel('Frequency')
```

```
plt.title('Distribution of Iris Features')
plt.legend()
plt.show()
```

OUTPUT



Class Performance	
Record	
Viva	
Total	

RESULT

Thus the program is executed for reading data from text files, Excel and the web and exploring various commands for doing descriptive analytics on the Iris data set using python.

EX.NO:
DATE:

Visualizing geographic data with Basemap

AIM

Visualizing geographic data with Basemap in Python using Basemap tool for plotting geographic data on maps.

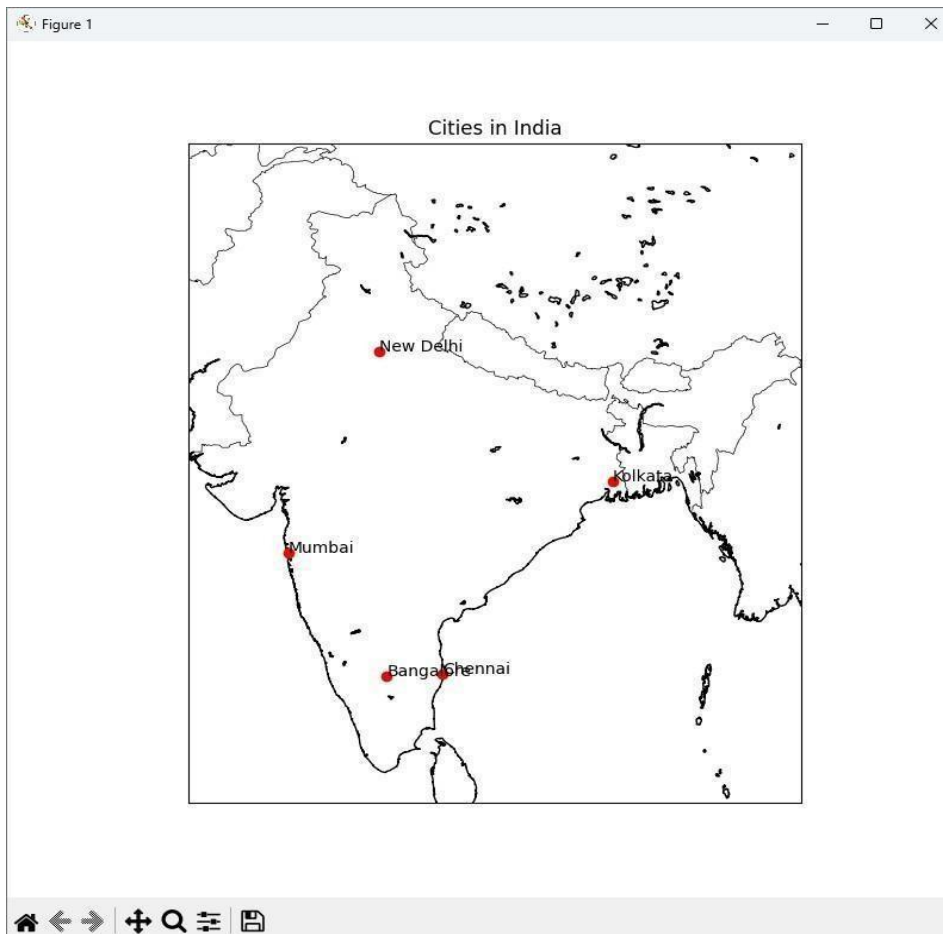
ALGORITHM

1. Install Basemap: First, make sure you have Basemap installed in your Python environment.
2. Import Libraries: Import the necessary libraries, including Basemap, Matplotlib, and NumPy
3. Create Basemap Instance: Create a Basemap instance by specifying the map projection, projection parameters, and other settings.
4. Plot Data: Plot your geographic data on the map using appropriate methods provided by Basemap.
5. Add Map Features: Add features such as coastlines, countries, and meridians to the map.
6. Customize Plot: Customize the plot by adding a title, legend, or any other annotations.
7. Show or Save Plot: Finally, display the plot or save it to a file.

PROGRAM

```
import matplotlib.pyplot as plt
from mpl_toolkits.basemap import Basemap
map = Basemap(llcrnrlon=68.1, llcrnrlat=6.5, urcrnrlon=97.4, urcrnrlat=37.6, resolution='i',
projection='merc')
map.drawcountries()
map.drawstates()
cities = {
    'New Delhi': (28.6139, 77.2090),
    'Mumbai': (19.0760, 72.8777),
    'Bangalore': (12.9716, 77.5946),
    'Chennai': (13.0827, 80.2707),
    'Kolkata': (22.5726, 88.3639)}
for city, (lat, lon) in cities.items():
    x, y = map(lon, lat)
    plt.plot(x, y, 'ro') # Plot cities as red circles
    plt.text(x, y, city, fontsize=10)
plt.title('Cities in India')
plt.show()
```

OUTPUT



Class Performance	
Record	
Viva	
Total	

RESULT

Thus the execution of visualization of basemap is done with python code for displaying India map.

EX.NO:
DATE:

Comparative statistical and predictive analysis of UCI and pima indians diabetes datasets

Use the diabetes data set from UCI and Pima Indians Diabetes data set for performing the following:

- a. Univariate analysis: Frequency, Mean, Median, Mode, Variance, Standard Deviation, Skewness and Kurtosis.
- b. Bivariate analysis: Linear and logistic regression modeling
- c. Multiple Regression analysis
- d. Also compare the results of the above analysis for the two data sets

AIM

To perform univariate, bivariate (linear & logistic) and multiple regression analyses on the UCI Diabetes and Pima Indians Diabetes datasets and compare the statistical outcomes to gain insights into diabetes prediction.

ALGORITHM

1. **Import & Clean Data** – Load both datasets and handle missing values.
2. **Univariate Analysis** – Calculate basic stats: mean, median, mode, std, skewness, kurtosis.
3. **Linear Regression** – Predict continuous variable using one feature.
4. **Logistic Regression** – Predict binary outcome using one feature.
5. **Multiple Regression** – Use multiple features to predict outcome.
6. **Compare Results** – Analyze and compare findings from both datasets.

PROGRAM

```
a.  
import numpy as np  
import pandas as pd  
from pandas import read_csv  
import seaborn as sns  
import matplotlib.pyplot as plt  
import statistics  
import scipy  
from scipy.stats import skew  
from scipy.stats import kurtosis  
df=pd.read_csv(r"C:\Users\ybkal\DSclab\diabetes.csv")  
df.shape  
print(df.head())  
sns.histplot(df['Age'])
```

```

plt.show()
print("\n Mean:\n",np.mean(df))
print("\n Standard deviation:\n",np.std(df,axis = 1))
print("\n Variance:\n",np.var(df,axis = 1))
print("\n Median:\n",np.median(df))
print("\n Mode:\n",statistics.mode(df))
unique,frequency = np.unique(df, return_counts = True)
count = np.asarray((unique,frequency))
print("Unique Values and frequency are :\n",count)
print("\n\n Skewness:\n")
print(skew(df, axis=0, bias=True))
print(kurtosis(df, axis=0, bias=True))

```

OUTPUT

```

--- Univariate Analysis for column: 'Outcome' ---

```

```

Frequency (Value Counts):

```

```

Outcome

```

```

0      500

```

```

1      268

```

```

Mean: 0.3489583333333333

```

```

Median: 0.0

```

```

Mode: 0

```

```

Variance: 0.22748261625380273

```

```

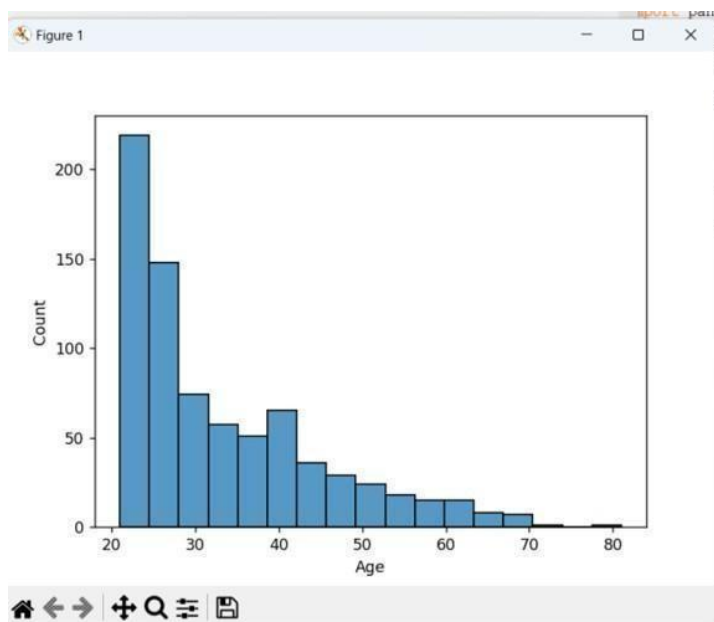
Standard Deviation: 0.47695137724279896

```

```

Skewness: 0.6337757030614577

```



b.

```
import pandas as pd
import numpy as np
from scipy.stats import mode, skew, kurtosis
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import mean_squared_error, accuracy_score
from sklearn.preprocessing import StandardScaler
uci_diabetes_data = pd.read_csv("diabetes.csv")
pima_diabetes_data = pd.read_csv("diabetes.csv")
def univariate_analysis(data):
    print("Univariate Analysis:")
    print("Frequency:")
    print(data['Outcome'].value_counts())
    print("Mean:", data['Outcome'].mean())
    print("Median:", data['Outcome'].median())
    print("Mode:", mode(data['Outcome'])[0])
    print("Variance:", data['Outcome'].var())
    print("Standard Deviation:", data['Outcome'].std())
    print("Skewness:", skew(data['Outcome']))
    print("Kurtosis:", kurtosis(data['Outcome']))
def bivariate_analysis_linear_regression(data):
    print("\nBivariate Analysis - Linear Regression:")
    X = data.drop(columns=['Outcome'])
    y = data['Outcome']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
    model = LinearRegression()
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    mse = mean_squared_error(y_test, y_pred)
    print("Mean Squared Error:", mse)
```

OUTPUT

```
Analysis for column: 'DiabetesPedigreeFunction'
Descriptive Statistics:
count      768.000000
mean       0.471876
std        0.331329
min        0.078000
25%        0.243750
50%        0.372500
75%        0.626250
max        2.420000
Name: DiabetesPedigreeFunction, dtype: float64
Skewness: 1.9161592037386292
Kurtosis: 5.550792047551203

Analysis for column: 'Age'
Descriptive Statistics:
count      768.000000
mean       33.240885
std        11.760232
min        21.000000
25%        24.000000
50%        29.000000
75%        41.000000
max        81.000000
Name: Age, dtype: float64
Skewness: 1.127389259531697
Kurtosis: 0.6311769413798585

Analysis for column: 'Outcome'
Descriptive Statistics:
count      768.000000
mean       0.348958
std        0.476951
min        0.000000
25%        0.000000
50%        0.000000
75%        1.000000
max        1.000000
Name: Outcome, dtype: float64
Skewness: 0.6337757030614577
Kurtosis: -1.5983283582089547

Bivariate Analysis - Linear Regression:
Mean Squared Error: 0.17104527280850096
```

C.

```
import pandas as pd
import numpy as np
from scipy.stats import mode, skew, kurtosis
from sklearn.model_selection import train_test_split
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import mean_squared_error, accuracy_score
from sklearn.preprocessing import StandardScaler
def bivariate_analysis_logistic_regression(data):
    print("\nBivariate Analysis - Logistic Regression:")
    X = data.drop(columns=['Outcome'])
    y = data['Outcome']
```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
model = LogisticRegression()
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

```

OUTPUT

```

RESTART: C:/Users/admin/ADD2-3.py =====
=====
Downloading dataset from Kaggle...
Dataset loaded successfully.

Bivariate Analysis - Logistic Regression:
Accuracy: 0.7532467532467533
|

```

d.

```

import pandas as pd
import numpy as np
from scipy.stats import mode, skew, kurtosis
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import mean_squared_error, accuracy_score
from sklearn.preprocessing import StandardScaler
def multiple_regression_analysis(data):
    print("\nMultiple Regression Analysis:")
    X = data.drop(columns=['Outcome'])
    y = data['Outcome']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
    model = LinearRegression()
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    mse = mean_squared_error(y_test, y_pred)
    print("Mean Squared Error:", mse)
print("Analysis on UCI Diabetes Dataset:")
univariate_analysis(uci_diabetes_data)

```

```

bivariate_analysis_linear_regression(uci_diabetes_data)
bivariate_analysis_logistic_regression(uci_diabetes_data)
multiple_regression_analysis(uci_diabetes_data)
print("\nAnalysis on Pima Indians Diabetes Dataset:")
univariate_analysis(pima_diabetes_data)
bivariate_analysis_linear_regression(pima_diabetes_data)
bivariate_analysis_logistic_regression(pima_diabetes_data)
multiple_regression_analysis(pima_diabetes_data)

```

OUTPUT

```

Analysis on Pima Indians Diabetes Dataset:
Univariate Analysis:
Frequency:
Outcome
0      500
1      268
Name: count, dtype: int64
Mean: 0.3489583333333333
Median: 0.0
Mode: 0
Variance: 0.22748261625380273
Standard Deviation: 0.47695137724279896
Skewness: 0.6337757030614577
Kurtosis: -1.5983283582089547

Bivariate Analysis - Linear Regression:
Mean Squared Error: 0.17104527280850101

Bivariate Analysis - Logistic Regression:
Accuracy: 0.7532467532467533

Multiple Regression Analysis:
Mean Squared Error: 0.17104527280850101

```

Class Performance	
Record	
Viva	
Total	

RESULT

Thus the python code is executed to do the Univariate , Bivariate analysis for the dataset given.

EX.NO:
DATE:

Visual Exploration of UCI Diabetes dataset using Advanced plotting techniques

Apply and explore various plotting functions on UCI data sets.

- a. Normal curves
- b. Density and contour plots
- c. Correlation and scatter plots
- d. Histograms
- e. Three dimensional plotting

AIM

To apply and explore various data visualization techniques on the UCI Diabetes dataset for better understanding of data distribution and relationships.

ALGORITHM

1. **Load Dataset** – Import the UCI Diabetes dataset using pandas.
2. **Normal Curves** – Plot normal distribution curves for numerical features.
3. **Density & Contour Plots** – Use `sns.kdeplot` and `sns.kdeplot` with `fill=True` for contours.
4. **Correlation & Scatter Plots** – Use `sns.heatmap` for correlation and `sns.pairplot` or `plt.scatter` for scatter plots.
5. **Histograms** – Plot histograms using `plt.hist()` or `sns.histplot()`.
6. **3D Plotting** – Use `mpl_toolkits.mplot3d` or `plotly` to create 3D scatter or surface plots.

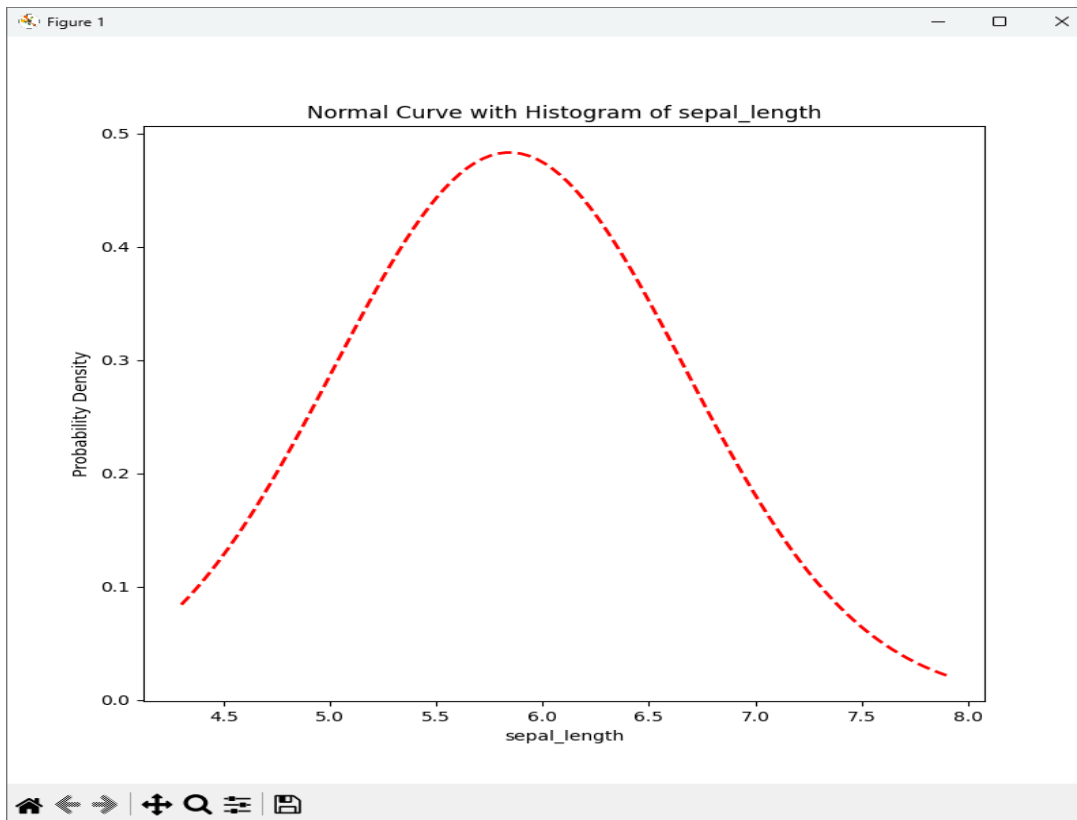
PROGRAM

a.

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import norm
dataset_url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
column_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'class']
df = pd.read_csv(dataset_url, names=column_names)
feature = 'sepal_length'
data = df[feature]
mean = np.mean(data)
std_dev = np.std(data)
x = np.linspace(min(data), max(data), 100)
y = norm.pdf(x, mean, std_dev)
plt.plot(x, y, 'r--', linewidth=2)
plt.xlabel(feature)
```

```
plt.ylabel('Probability Density')
plt.title('Normal Curve with Histogram of {}'.format(feature))
plt.show()
```

OUTPUT

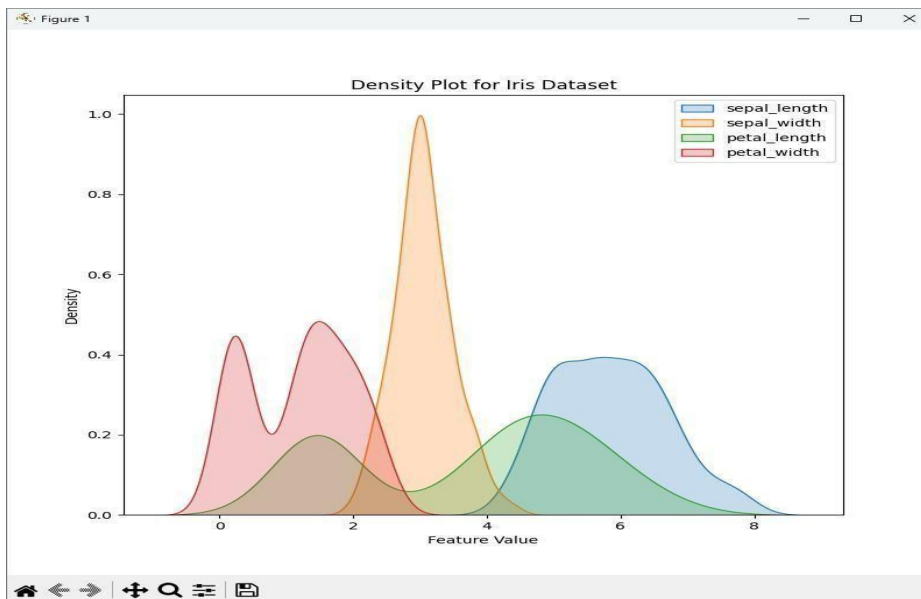


b.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
column_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'class']
iris_df = pd.read_csv(url, names=column_names)
plt.figure(figsize=(10, 6))
for feature in iris_df.columns[:-1]: # Excluding the last column which is the class
    sns.kdeplot(iris_df[feature], shade=True, label=feature)
plt.title('Density Plot for Iris Dataset')
plt.xlabel('Feature Value')
plt.ylabel('Density')
plt.legend()
plt.show()
```

```
plt.figure(figsize=(10, 6))
sns.kdeplot(data=iris_df[['sepal_length', 'sepal_width']], cmap="viridis", fill=True)
plt.title('Contour Plot for Sepal Length vs Sepal Width')
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.show()
```

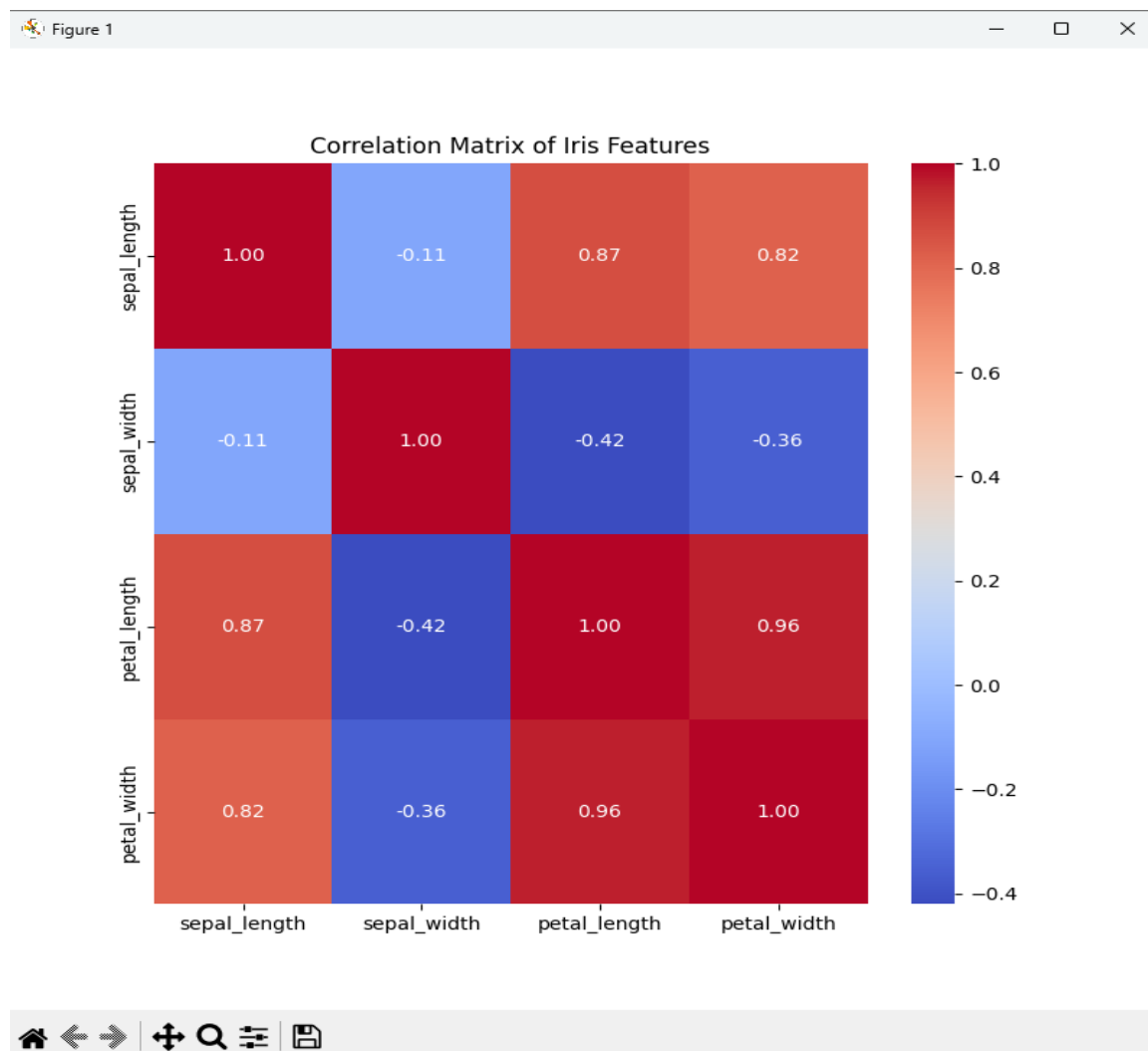
OUTPUT



c.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
column_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'class']
iris_df = pd.read_csv(url, names=column_names)
numeric_df = iris_df.drop(columns=['class'])
correlation_matrix = numeric_df.corr()
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix of Iris Features')
plt.show()
```

OUTPUT

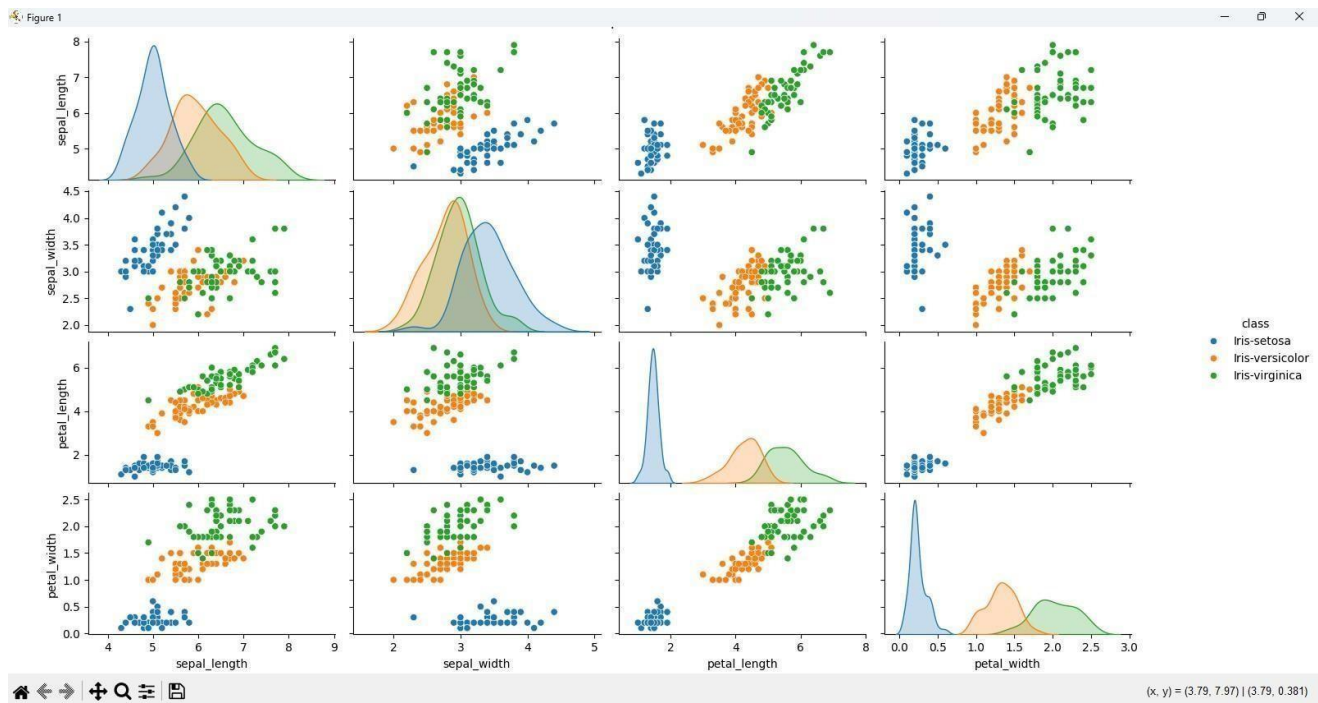


d.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
column_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'class']
iris_df = pd.read_csv(url, names=column_names)
sns.pairplot(iris_df, hue='class', diag_kind='kde')
plt.suptitle('Pairplot of Iris Features', y=1.02)
plt.show()
```

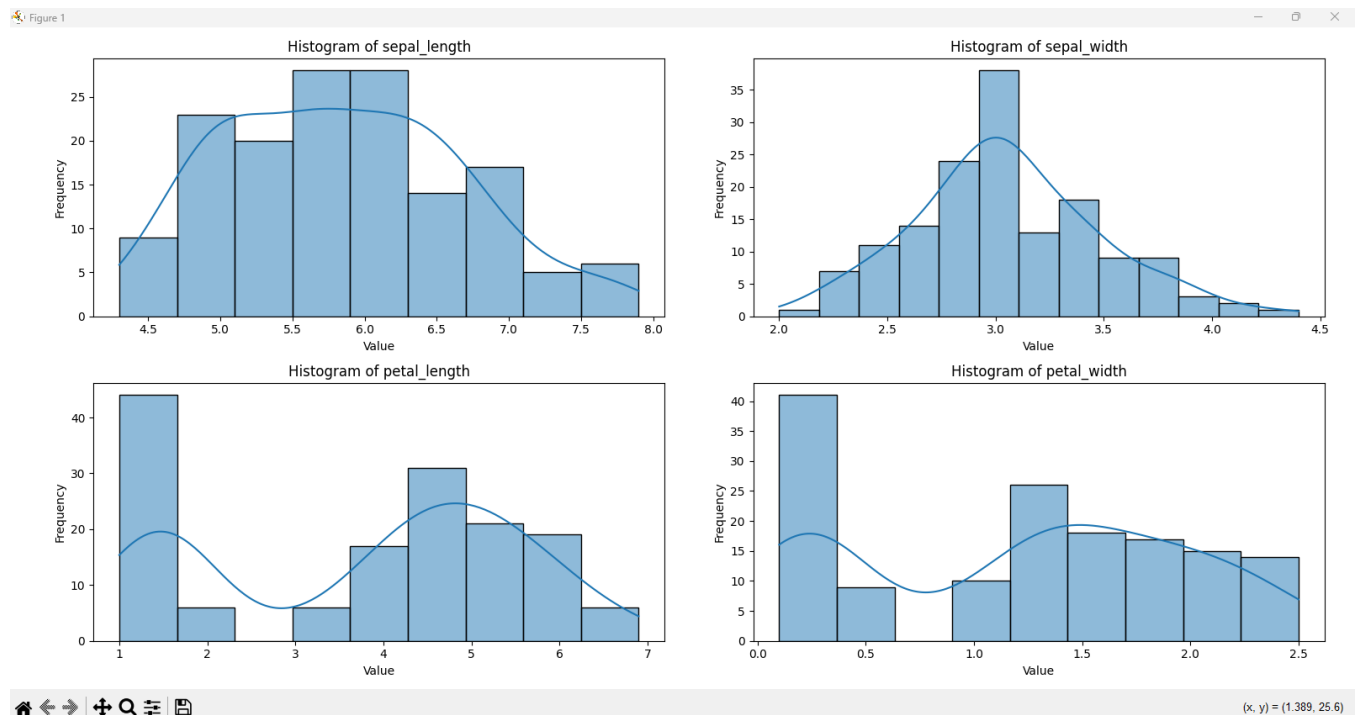
OUTPUT



e.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
column_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'class']
iris_df = pd.read_csv(url, names=column_names)
plt.figure(figsize=(10, 8))
for i, feature in enumerate(iris_df.columns[:-1]):
    plt.subplot(2, 2, i+1)
    sns.histplot(iris_df[feature], kde=True)
    plt.title('Histogram of {}'.format(feature))
    plt.xlabel('Value')
    plt.ylabel('Frequency')
plt.tight_layout()
plt.show()
```

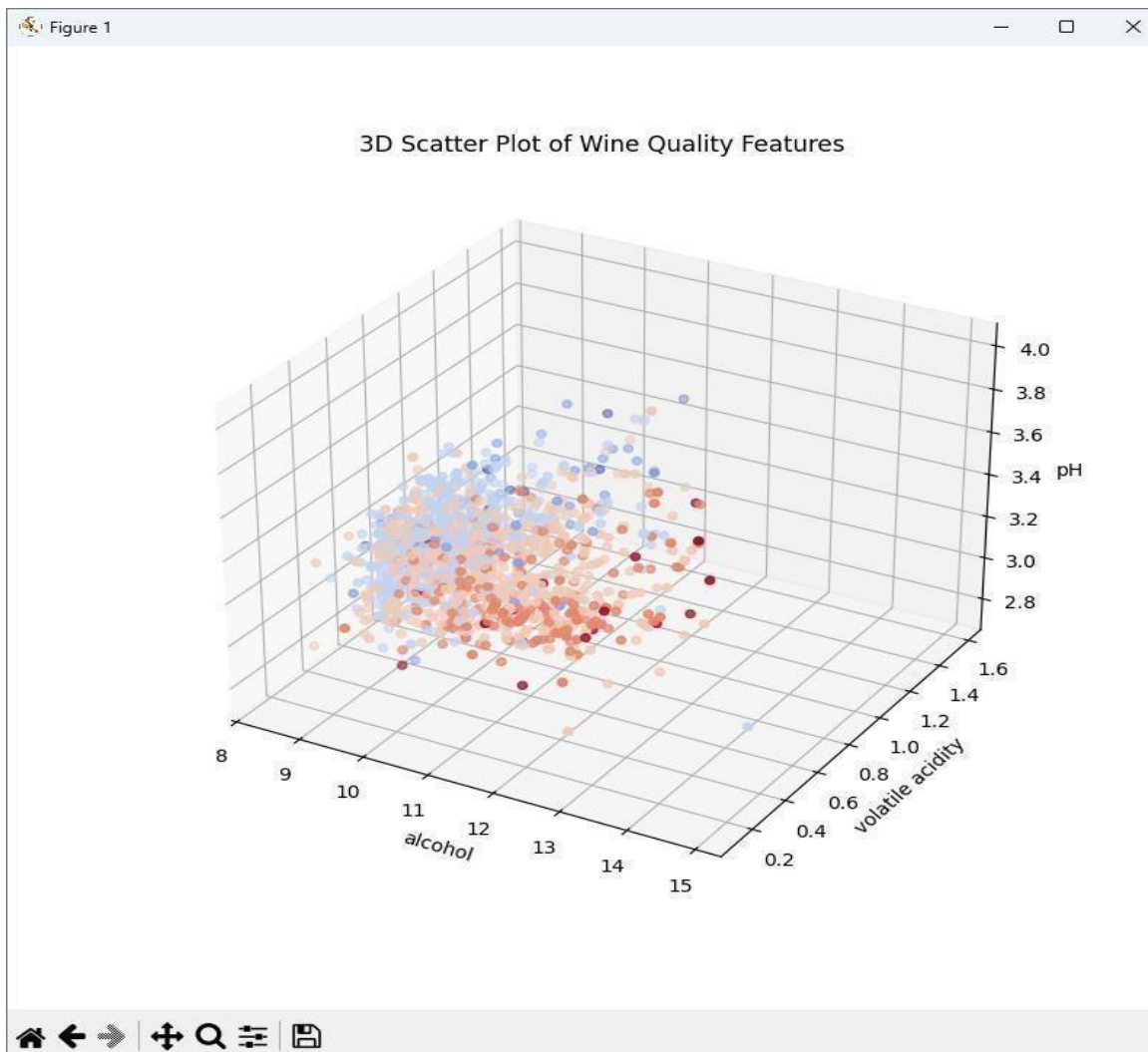
OUTPUT



f.

```
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv"
wine_df = pd.read_csv(url, sep=';')
feature1 = 'alcohol'
feature2 = 'volatile acidity'
feature3 = 'pH'
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(wine_df[feature1], wine_df[feature2], wine_df[feature3], c=wine_df['quality'],
cmap='coolwarm')
ax.set_xlabel(feature1)
ax.set_ylabel(feature2)
ax.set_zlabel(feature3)
ax.set_title('3D Scatter Plot of Wine Quality Features')
plt.show()
```

OUTPUT



Class Performance	
Record	
Viva	
Total	

RESULT

Thus the python code is executed to Apply and explore various plotting functions on UCI data set.